

# Chaînes de Markov cachées et application en finance

Sous la supervision de *Joseph Rynkiewicz et Jean-Marc Bardet*.

Par Marius Requillart et Maxime Delpech.

2026

# SOMMAIRE

## **I) Introduction**

## **II) Définition et propriétés**

### 2.1) Définitions

2.1.1) Variables aléatoires observables et cachées.

2.1.2) Propriétés.

### 2.2) Définition chaîne de Markov cachée.

2.2.1) Variables aléatoires observables et cachées.

2.2.2) Lois jointes et vraisemblance.

2.2.3) Exemple global d'une chaîne de Markov cachée.

2.2.4) Remarques sur les observations.

## **III) Trois grandes problématiques des HMM**

1) Estimation de la vraisemblance et forward algorithm.

2) Décodage et Viterbi algorithm.

3) Apprentissage et Baum–Welch algorithm.

## **IV) Limites des algorithmes et mesure de la qualité des estimations**

1) Limites des algorithmes

2) Mesure de la qualité des estimations de l'algorithme de Forward.

3) Mesure de la qualité des estimations de l'algorithme de Viterbi.

4) Mesure de la qualité des estimations de l'algorithme de Baum–Welch.

## **IV) Application aux séries temporelles financières**

# 1 Introduction

Les séries temporelles financières occupent une place centrale en finance quantitative, car elles permettent d'analyser l'évolution des prix d'actifs, des rendements, de la volatilité ou encore des volumes échangés au cours du temps. Cependant, ces séries présentent souvent des comportements complexes : elles sont bruitées, irrégulières, parfois instables, et peuvent être marquées par des changements soudains de dynamique. Une même série financière peut ainsi alterner entre des périodes relativement calmes et des périodes de forte incertitude, ce qui rend son analyse particulièrement délicate.

Les chaînes de Markov cachées, ou Hidden Markov Models, constituent un cadre probabiliste adapté pour modéliser ce type de phénomènes. Leur principe repose sur l'existence d'un état latent, non directement observable, qui évolue au cours du temps selon une chaîne de Markov. Les observations disponibles, comme les rendements financiers, sont alors supposées dépendre de cet état caché.

L'intérêt de ces modèles réside dans leur capacité à faire apparaître une structure cachée derrière les données observées. Ils permettent non seulement de détecter des changements de régime, mais aussi de donner une interprétation économique et financière aux résultats obtenus. Les états cachés identifiés peuvent par exemple être associés à des phases de croissance, de stress financier, de forte incertitude ou de retour à une situation plus stable. Ainsi, l'analyse ne se limite pas à une classification statistique des observations : elle peut aussi aider à relier les régimes détectés à des événements de marché, à des changements d'anticipations ou à des variations du niveau de risque.

Dans ce travail, nous étudierons les chaînes de Markov cachées et leurs applications aux séries temporelles financières. Nous présenterons d'abord le cadre théorique de ces modèles, avant d'aborder les principaux algorithmes d'inférence et d'estimation, notamment les algorithmes de Viterbi et de Baum-Welch. Nous analyserons ensuite leur utilisation pour identifier différents régimes de marché et mieux comprendre l'évolution des séries financières.

## 2 Définitions et propriétés

Toutes les variables aléatoires de ce travail sont définies sur l'espace probabilisé  $(\Omega, \mathcal{F}, \mathbb{P})$ .

### 2.1 Définitions et propriétés d'un chaîne de Markov

#### 2.1.1 Définition

##### Définition d'une chaîne de Markov :

Soit  $E$  un ensemble fini ou dénombrable, appelé *espace d'états*. Soit  $(X_n)_{n \geq 0}$  un *processus* à valeur dans  $E$ . On dit que  $(X_n)_{n \geq 0}$  est une *chaîne de Markov* d'ordre 1, si :

Pour tout  $n \geq 0$ , pour tout  $y \in E$ , pour tout  $x_0, \dots, x_{n-1}, x \in E$  tel que  $\mathbb{P}(X_0 = x_0, \dots, X_{n-1} = y, X_n = x) > 0$ , on a :

$$\mathbb{P}(X_{n+1} = y \mid X_n = x, X_{n-1} = x_{n-1}, \dots, X_0 = x_0) = \mathbb{P}(X_{n+1} = y \mid X_n = x).$$

On pose  $\pi_0$  la distribution initiale, i.e., la loi de  $X_0$ .

De plus, s'il existe  $A : E \times E \rightarrow [0, 1]$  tel que  $(x, y) \mapsto a_{x,y} = \mathbb{P}(X_{n+1} = y \mid X_n = x)$ , pour tout  $n \geq 0$ , pour tout  $x, y \in E$  tel que  $\mathbb{P}(X_n = x) > 0$ . On dit alors que  $(X_n)_{n \geq 0}$  est une chaîne de Markov d'ordre 1 *homogène*. Dans ce cas,  $A$  est appelé *matrice de transition* du processus  $(X_n)_{n \geq 0}$ , avec  $a_{x,y} > 0$  et  $\sum_{y \in E} a_{x,y} = 1$ , pour tout  $x, y \in E$ .

Dans ce travail, nous ne considérerons qu'uniquement les chaînes de Markov d'ordre 1 homogènes, c'est pourquoi nous les appellerons juste chaînes de Markov.

#### 2.1.2 Propriétés

##### • Loi d'une trajectoire markovienne :

Soit  $(X_n)_{n \geq 0}$  une chaîne de Markov de matrice de transition  $P$  et de distribution initiale  $\pi_0$ . On a, Pour tout  $n \geq 0$ , pour tout  $x_0, \dots, x_n \in E$ ,

$$\mathbb{P}(X_0 = x_0, \dots, X_n = x_n) = \pi_0(x_0) \mathbb{P}(x_0, x_1) \dots \mathbb{P}(x_{n-1}, x_n)$$

i.e. la loi d'une chaîne de Markov est entièrement caractérisée par sa distribution initiale et sa matrice de transition.

##### • Propriété de Markov :

Soit  $n \geq 0$ ,  $\pi_0$  une mesure de probabilité sur  $E$  et  $x \in E$  tel que  $\mathbb{P}_{\pi_0}(X_n = x) > 0$ .

Alors, pour tout  $A \in E^n$ ,  $B \in E^m$ ,  $m \geq 0$ , on a :

$$\mathbb{P}_{\pi_0}((X_0, \dots, X_{n-1}) \in A, (X_{n+1}, \dots, X_{n+m}) \in B \mid X_n = x) = \mathbb{P}_{\pi_0}((X_0, \dots, X_{n-1}) \in A \mid X_n = x) \\ \times \mathbb{P}_{\pi_0}((X_{n+1}, \dots, X_{n+m}) \in B \mid X_n = x).$$

i.e. le passé et le futur d'un moment  $n$  d'une chaîne de Markov sont indépendants entre eux.

### Quelques propriétés utiles :

Pour tout  $x, y \in E$ , on a :

$$\mathbb{P}_x(X_n = y) = P^n(x, y).$$

De plus, pour toute mesure de probabilité  $\mu$  sur  $E$ , et pour tout  $y \in E$ , on a :

$$\mathbb{P}_\mu(X_n = y) = \sum_{x \in E} \mu(x) P^n(x, y) = (\mu P^n)(y).$$

## 2.2 Définition d'une chaîne de Markov cachée

Les chaîne de Markov cachées (HMM) sont des modèles statistique utilisées dans un situation où on a des états latent qui ne sont pas directement observable. Les données observables sont générées par ces état suivant une loi de probabilité conditionnelle.

### 2.2.1 Variables aléatoires observables et cachées

**Définition.** Les états cachés sont modélisés par un processus de Markov  $(X_t)_{t \in \mathbb{N}}$  à valeurs dans un ensemble fini  $E = \{1, \dots, N\}$ . On note  $A = (a_{ij})$  la matrice de transition définie par :

$$a_{ij} = \mathbb{P}(X_{t+1} = j \mid X_t = i).$$

**Définition.** Les observations sont modélisées par un processus  $(Y_t)_{t \in \mathbb{N}}$  à valeurs dans un ensemble  $O$ . Conditionnellement aux états cachés, les variables  $(Y_t)$  sont indépendantes et vérifient :

$$\mathbb{P}(Y_t = y \mid X_t = i) = b_i(y),$$

où  $B = (b_i(y))$  est la matrice (ou famille) d'émission.

**Définition.** Un modèle de chaîne de Markov cachée est défini par le triplet  $(A, B, \pi)$  où :

- $A$  est la matrice de transition des états cachés,
- $B$  est la loi d'émission des observations,
- $\pi$  est la distribution initiale, avec  $\pi_i = \mathbb{P}(X_1 = i)$ .

### 2.2.2 Loi jointes et vraisemblance

**Propriété :** Soit  $X = (X_1, \dots, X_T)$  la séquence d'états cachés et  $Y = (Y_1, \dots, Y_T)$  la séquence d'observations, la loi des observations conditionnellement à la séquence d'états cachés est :

$$\mathbb{P}(Y | X) = \prod_{t=1}^T \mathbb{P}(Y_t | X_t)$$

**Preuve :**

$$\mathbb{P}(Y | X) = \prod_{t=1}^T \mathbb{P}(Y_t | X_1, \dots, X_T, Y_1, \dots, Y_{t-1}) = \prod_{t=1}^T \mathbb{P}(Y_t | X_t)$$

**Propriété :** Soit  $X = (X_1, \dots, X_T)$  la séquence d'états cachés et  $Y = (Y_1, \dots, Y_T)$  la séquence d'observations, la loi jointes des observations et des états cachés est donnée par :

$$\mathbb{P}(Y, X) = \pi_{X_1} \prod_{t=2}^T \mathbb{P}(X_t | X_{t-1}) \prod_{t=1}^T \mathbb{P}(Y_t | X_t)$$

**Preuve :**

$$\begin{aligned} \mathbb{P}(X, Y) &= \mathbb{P}(X_1, Y_1) \prod_{t=2}^T \mathbb{P}(X_t, Y_t | X_1, \dots, X_{t-1}, Y_1, \dots, Y_{t-2}) \\ &= \mathbb{P}(X_1) \mathbb{P}(Y_1 | X_1) \prod_{t=2}^T \mathbb{P}(X_t | X_1, \dots, X_{t-1}, Y_1, \dots, Y_{t-2}) \mathbb{P}(Y_t | X_1, \dots, X_{t-1}, Y_1, \dots, Y_{t-2}) \\ &= \mathbb{P}(X_1) \mathbb{P}(Y_1 | X_1) \prod_{t=2}^T \mathbb{P}(X_t | X_{t-1}) \mathbb{P}(Y_t | X_t) \\ &= \mathbb{P}(X_1) \prod_{t=2}^T \mathbb{P}(X_t | X_{t-1}) \prod_{t=1}^T \mathbb{P}(Y_t | X_t) \\ &= \pi_{X_1} \prod_{t=2}^T \mathbb{P}(X_t | X_{t-1}) \prod_{t=1}^T \mathbb{P}(Y_t | X_t) \end{aligned}$$

**Remarque :** La loi des observations à chaque instant  $t$  est dépend de l'état caché à cet instant et pas des états d'avant.

**Exemple :** Soit un modèle de chaîne de Markov cachée avec deux états  $A$  et  $B$ . En supposant que les observation sont gaussienne conditionnellement à l'état caché, on a :

$$\mathcal{L}(Y_t | X_t = A) \sim \mathcal{N}(\mu_A, \sigma_A^2)$$

$$\mathcal{L}(Y_t | X_t = B) \sim \mathcal{N}(\mu_B, \sigma_B^2)$$

**Remarque :**  $X_t$  agit comme un selecteur de distribution pour  $Y_t$ .

### 2.2.3 Exemple de globale d'une chaîne de Markov cachée

Prenons un exemple simple d'une chaîne de Markov cachée avec deux états  $C$  et  $V$  (pour "calme" et "volatile") et des observations gaussiennes conditionnelles à ces états.

Soit  $E = \{C, V\}$  l'ensemble d'états, on note  $\pi$  la distribution initiale, par exemple  $\pi_0 = (0.5, 0.5)$ , indiquant que le processus commence avec une probabilité égale dans les états  $C$  et  $V$ .

La matrice de transition  $A$  peut être définie comme suit :

$$A = \begin{pmatrix} 0.9 & 0.1 \\ 0.2 & 0.8 \end{pmatrix}$$

Avec,  $a_{ij} = \mathbb{P}(X_{t+1} = j \mid X_t = i)$ .

La famille de densités paramétriques  $B$ , peut être définie par les distributions gaussiennes conditionnelles à chaque état :

$$B = \begin{cases} \mathcal{L}(Y_t \mid X_t = C) \sim \mathcal{N}(\mu_C, \sigma_C^2) \\ \mathcal{L}(Y_t \mid X_t = V) \sim \mathcal{N}(\mu_V, \sigma_V^2) \end{cases}$$

**NB :** La matrice de transition  $A$  n'est pas connue dans la plupart des applications, elle doit être estimé à partir des données d'observations. Par exemple avec l'algorithme de Baum-Welch que nous verrons plus tard.

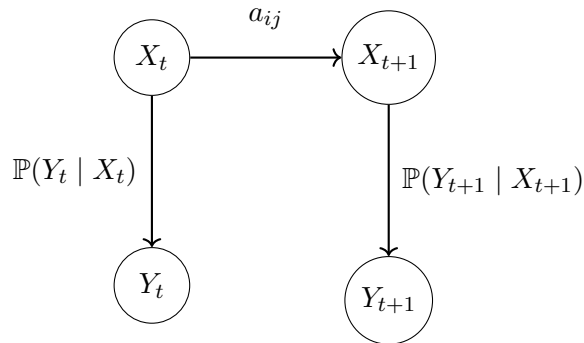


FIGURE 1 – Modèle de Markov caché (HMM)

### 2.2.4 Remarques sur les observations

**Propriété :** La probabilité totale des observation  $(Y_t)_t$  est donnée par :

$$\mathbb{P}(Y) = \sum_X \mathbb{P}(Y, X) = \sum \mathbb{P}(Y | X) \mathbb{P}(X)$$

**Propriété :** Dans un HMM non dégénéré, les observations  $(Y_t)_t$  n'est pas un processus de Markov, même si les états cachés  $(X_t)_t$  le sont.

**Preuve :**

Si le processus d'observations  $(Y_t)_t$  était markovien. Alors on aurait

$$\mathbb{P}(Y_{n+1} | Y_1, \dots, Y_n) = \mathbb{P}(Y_{n+1} | Y_n).$$

Or, pour  $K$  états cachés, on a

$$\begin{aligned} \mathbb{P}(Y_{n+1} | Y_1, \dots, Y_n) &= \sum_{i=1}^K \mathbb{P}(Y_{n+1}, X_{n+1} = i | Y_1, \dots, Y_n) \\ &= \sum_{i=1}^K \mathbb{P}(Y_{n+1} | X_{n+1} = i, Y_1, \dots, Y_n) \mathbb{P}(X_{n+1} = i | Y_1, \dots, Y_n) \\ &= \sum_{i=1}^K \mathbb{P}(Y_{n+1} | X_{n+1} = i) \mathbb{P}(X_{n+1} = i | Y_1, \dots, Y_n). \end{aligned}$$

or,

$$\begin{aligned} \mathbb{P}(X_{n+1} = i | Y_1, \dots, Y_n) &= \sum_{j=1}^K \mathbb{P}(X_{n+1} = i, X_n = j | Y_1, \dots, Y_n) \\ &= \sum_{j=1}^K \mathbb{P}(X_{n+1} = i | X_n = j, Y_1, \dots, Y_n) \mathbb{P}(X_n = j | Y_1, \dots, Y_n) \\ &= \sum_{j=1}^K \mathbb{P}(X_{n+1} = i | X_n = j) \mathbb{P}(X_n = j | Y_1, \dots, Y_n) \\ &= \sum_{j=1}^K a_{ji} \mathbb{P}(X_n = j | Y_1, \dots, Y_n). \end{aligned}$$



et,

$$\begin{aligned}
\mathbb{P}(X_n = j \mid Y_1, \dots, Y_n) &= \mathbb{P}(X_n = j, Y_1 \dots Y_n) / \mathbb{P}(Y_1 \dots Y_n) \\
&= \frac{\mathbb{P}(Y_n \mid X_n = j, Y_1 \dots Y_{n-1}) \mathbb{P}(X_n = j, Y_1 \dots Y_{n-1})}{\mathbb{P}(Y_1 \dots Y_n)} \\
&= \frac{\mathbb{P}(Y_n \mid X_n = j) \mathbb{P}(X_n = j \mid Y_1 \dots Y_{n-1}) \mathbb{P}(Y_1 \dots Y_{n-1})}{\mathbb{P}(Y_n \mid Y_1 \dots Y_{n-1}) \mathbb{P}(Y_1 \dots Y_{n-1})} \\
&= \frac{\mathbb{P}(Y_n \mid X_n = j) \mathbb{P}(X_n = j \mid Y_1 \dots Y_{n-1})}{\mathbb{P}(Y_n \mid Y_1 \dots Y_{n-1})}.
\end{aligned}$$

on a donc :

$$\mathbb{P}(Y_{n+1} \mid Y_1 \dots Y_n) = \sum_{i=1}^K \sum_{j=1}^K a_{ji} \frac{\mathbb{P}(Y_{n+1} \mid X_{n+1} = i) \mathbb{P}(Y_n \mid X_n = j) \mathbb{P}(X_n = j \mid Y_1 \dots Y_{n-1})}{\mathbb{P}(Y_n \mid Y_1 \dots Y_{n-1})}.$$

On procède de la même manière pour calculer  $\mathbb{P}(X_n = j \mid Y_1 \dots Y_{n-1})$  ce qui va descendre le degré de dépendance à  $Y_{n-1}$ . On continue ainsi de suite de manière récursive jusqu'à ce que le terme de dépendance à  $Y_1$  soit atteint.

Ainsi, le terme

$$\mathbb{P}(X_{n+1} = i \mid Y_1, \dots, Y_n)$$

dépend en général de tout le passé observé  $(Y_1, \dots, Y_n)$ , et pas seulement de  $Y_n$ .

Donc, en général,

$$\mathbb{P}(Y_{n+1} \mid Y_1, \dots, Y_n) \neq \mathbb{P}(Y_{n+1} \mid Y_n),$$

ce qui montre que le processus d'observations  $(Y_t)$  n'est pas markovien.

### 3 Trois grandes problématiques des HMM

#### 3.1 Estimation de la vraisemblance et forward algorithm

**Problématique :** Comment calculer efficacement la vraisemblance  $\mathbb{P}(Y)$  d'une séquence d'observations  $Y$  donnée un modèle de HMM  $(A, B, \pi)$  ?

On suppose que la matrice de transition  $A$ , la matrice d'émission  $B$  ainsi que la distribution initiale  $\pi$  sont connues.

On sait que la vraisemblance de  $Y$  est donnée par :

$$\mathbb{P}(Y) = \sum_X \mathbb{P}(Y, X) = \sum_X \mathbb{P}(Y | X) \mathbb{P}(X).$$

Cependant, pour  $N$  états cachés et une séquence d'observations de longueur  $T$ , le nombre de séquence cachées possibles est de  $N^T$ , ce qui rend le calcul direct d'une complexité exponentielle si on l'effectue de manière naïve.

La solution optimale est d'utiliser l'algorithme de forward qui permet de calculer  $\mathbb{P}(Y)$  avec un complexité de  $O(N^2T)$ .

Cet algorithme repose sur le concept de programmation dynamique et utilise une variable intermédiaire  $\alpha_t(i)$  qui représente la probabilité d'observer la séquence d'observation  $y_1 \dots y_t$  et d'être dans l'état caché  $i$  à l'instant  $t$ .

Formellement, on définit  $\alpha_t(i)$  comme :

$$\alpha_t(i) = \mathbb{P}(y_1, \dots, y_t, X_t = i).$$

---

**Algorithm 1** Algorithme Forward

---

**Require:** Matrice de transition  $A = (a_{ij})_{1 \leq i, j \leq N}$ , probabilités d'émission  $B = (b_i(y))_{1 \leq i \leq N}$ , distribution initiale  $\pi = (\pi_i)_{1 \leq i \leq N}$ , suite d'observations  $Y = (y_1, \dots, y_T)$

**Ensure:** Vraisemblance  $\mathbb{P}(Y)$

**Initialisation**

```
1: for  $i = 1, \dots, N$  do
2:    $\alpha_1(i) \leftarrow \pi_i b_i(y_1)$ 
3: end for
```

**Récursion**

```
4: for  $t = 2, \dots, T$  do
5:   for  $j = 1, \dots, N$  do
6:      $\alpha_t(j) \leftarrow b_j(y_t) \sum_{i=1}^N \alpha_{t-1}(i) a_{ij}$ 
7:   end for
8: end for
```

**Terminaison**

```
9:  $\mathbb{P}(Y) \leftarrow \sum_{i=1}^N \alpha_T(i)$ 
10: return  $\mathbb{P}(Y)$ 
```

---

avec  $a_{ij}$  la probabilité de transition de l'état  $i$  à l'état  $j$ ,  $b_j(y_t)$  la probabilité d'observer  $Y_t = y_t$  conditionnellement à être dans l'état  $j$  à l'instant  $t$ , et  $\pi_i$  la probabilité initiale d'être dans l'état  $i$ . L'algorithme de forward permet ainsi de calculer efficacement la vraisemblance d'une séquence d'observations donnée, en évitant le calcul direct de toutes les séquences d'états cachés possibles.

### Analyse de la complexité :

Lors de l'initialisation, on effectue  $N$  opérations de complexité  $O(1)$  pour calculer  $\alpha_1(i)$  pour  $i = 1, \dots, N$ , soit une complexité de  $O(N)$ .

Lors de la récursion, pour chaque instant  $t$  de 2 à  $T$ , on effectue  $N$  opérations pour calculer  $\alpha_t(j)$  pour  $j = 1, \dots, N$ . chaque calcul de  $\alpha_t(j)$  nécessite de faire une somme de  $N$  termes, ce qui correspond à une complexité de  $O(N^2)$  pour chaque instant  $t$ . Ainsi la complexité total de la récursion est de  $O(N^2T)$ .

Lors de la terminaison, on effectue une somme de  $N$  termes pour calculer  $\mathbb{P}(Y)$ , ce qui correspond à une complexité de  $O(N)$ .

En combinant ces différentes étapes, on obtient une complexité totale de l'algorithme de  $O(N^2T)$ , ce qui est beaucoup plus efficace que la complexité exponentielle  $O(N^T)$  du calcul brut.

Pour plus de clareté, on peut présenter l'algorithme de forward sous forme de treillis :

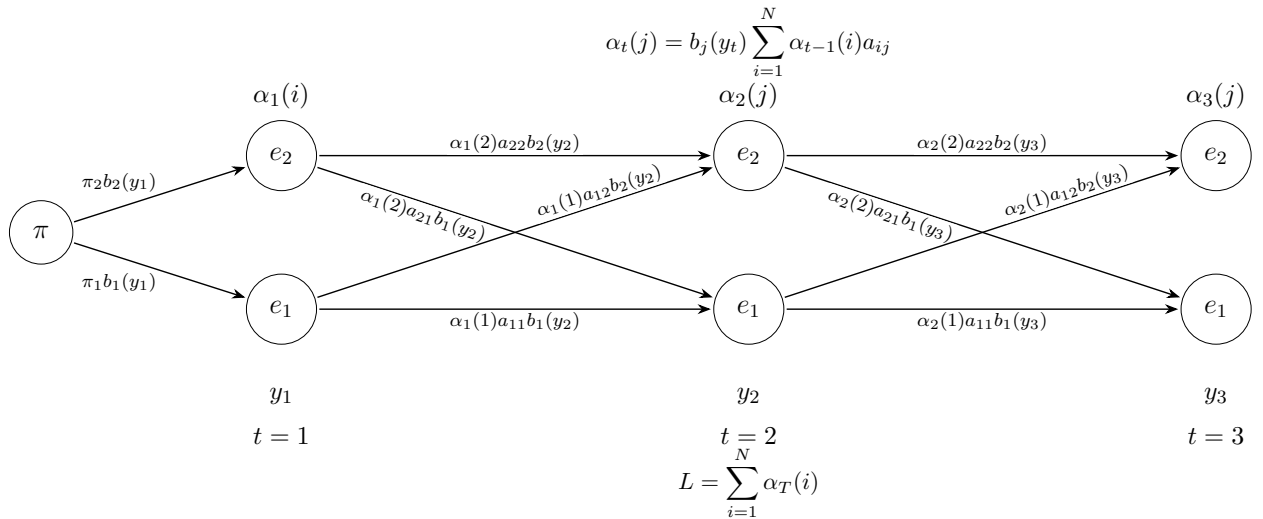


FIGURE 2 – Treillis de l'algorithme Forward

## 3.2 Decodding et Viterbi algorithm

Considérons un nouveau problème : le décodage d'une suite d'état d'après une série d'observations  $Y_1, \dots, Y_T$ . On essaye de découvrir la partie cachée du modèle.

i.e. Découvrir à posteriori la “vraie” séquence d’état  $X = (X_1, \dots, X_T)$  qui s’est produite pour générer une série d’observation, sachant la distribution initiale, la matrice de transition ainsi que les probabilités d’émissions.

Il paraît évident qu’il n’y a pas toujours de “vraie” séquence d’états, nous allons devoir instaurer des critères d’optimalité pour régler notre problème.

Une première approche qu’on pourrait qualifier de “Greedy” serait de choisir, à chaque instant  $t$ , l’état  $X_t$  tel que la probabilité d’observer  $Y_t$  est la plus grande. Ce critère d’optimalité maximise l’espérance du nombre d’états correctement décrypté un à un.

Cependant, cela peut amener plusieurs problèmes avec la séquence d’état qui en résulte. Comme le critère local ignore la matrice de transition, il peut proposer une transition de probabilité nulle. Or rien ne l’interdit dans notre critère, donc cette transition pourrait apparaître dans le résultat alors que celui-ci ne serait ainsi même pas réalisable.

Il va donc falloir trouver un critère prenant en compte la probabilité de réalisation d’une séquence d’états. On pourrait ainsi vouloir maximiser l’espérance du nombre de paires correctes d’états découvert, ou de triplet, etc. Cependant le critère le plus utilisé est le critère de trouver l’unique séquence d’états qui maximise la probabilité de voir nos observations sachant la distribution initiale, la matrice de transition ainsi que les probabilités d’émissions.

Le problème étant exponentiellement complexe, la solution est d’utiliser un algorithme de programmation dynamique nommé : Algorithme de Viterbi.

### 3.2.1 L’algorithme de Viterbi

Premièrement, on remarque que maximiser  $\mathbb{P}(X \mid Y, \lambda)$  est équivalent à maximiser  $\mathbb{P}(X, y \mid \lambda)$  car les observations sont fixées.

Afin de trouver la meilleure séquence d’états  $X = \{X_1, \dots, X_T\}$ , pour la séquence d’observations  $y = \{y_1, \dots, y_T\}$ , on définit la quantité :

$$\delta_t(i) = \max_{X_1, \dots, X_{t-1}} \mathbb{P}(X_1, X_2, \dots, X_t = i, y_1, y_2, \dots, y_t \mid \lambda)$$

i.e.  $\delta_t(i)$  est la probabilité du chemin le plus probable se terminant par l’état  $i$  au moment  $t$ .

Avec,

$$a_{ij} = \mathbb{P}(X_{t+1} = j \mid X_t = i)$$

et, si les observations sont discrètes,

$$b_j(y_t) = \mathbb{P}(Y_t = y_t \mid X_t = j)$$

ou, si les observations sont continues,

$$b_j(y_t) = f_{Y_t|X_t=j}(y_t)$$

Ainsi, par récurrence, on a que :

$$\delta_{t+1}(j) = \left[ \max_{1 \leq i \leq N} \delta_t(i) a_{ij} \right] b_j(y_{t+1})$$

**Problème :** En testant naïvement toutes les suites possibles d'états cachés, il y a  $N^T$  séquences possibles. Et pour calculer la probabilité de chaque séquence, il faut environ  $T$  opérations.

Ainsi on obtient une complexité de :

$$O(TN^T)$$

C'est exponentiel, donc inutilisable dès que  $T$  ou  $N$  devient grand.

Afin de limiter cette complexité, l'algorithme de Viterbi repose sur le principe du treillis.

### Principe du Treillis :

Structure en colonne où :

- chaque colonne correspond à un instant  $t$
- chaque ligne correspond à un état caché.
- chaque noeud  $(t, j)$  représente le fait d'être dans l'état  $j$  au temps  $t$ .
- chaque arête entre  $(t-1, i)$  et  $(t, j)$  représente une transition possible, pondéré par la proba  $a_{ij}$ .
- chaque noeud est également associé à la proba d'émission  $b_j(y_t)$ .

Ainsi, le problème de décodage revient à trouver le meilleur chemin dans ce treillis, depuis la première colonne jusqu'à la dernière. Chaque chemin correspond à une séquence d'états cachés possible. Le rôle de l'algorithme de Viterbi est précisément de trouver le chemin de probabilité maximale :

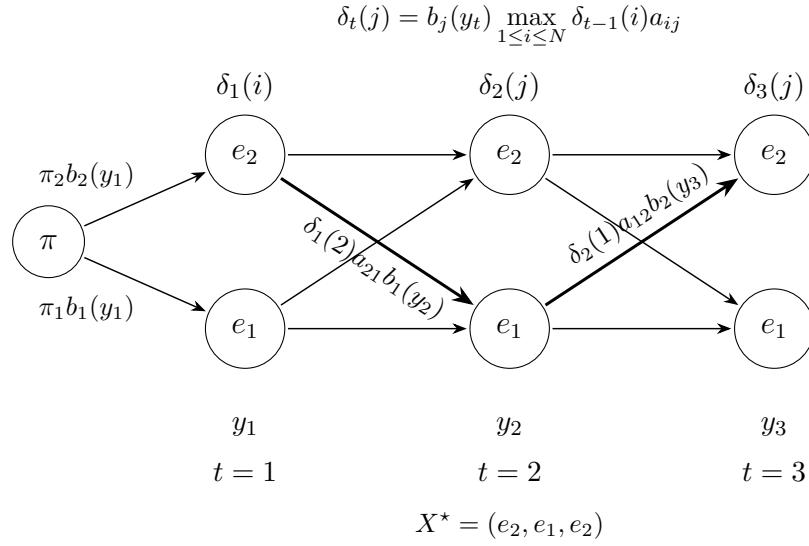


FIGURE 3 – Treillis de l’algorithme de Viterbi

L’intérêt du treillis est qu’il met en évidence une propriété fondamentale : pour retrouver la séquence d’état, il faut garder en mémoire l’argument qui maximise la récurrence. Nous allons le faire via la liste  $\psi_t(j)$ .

On peut donc suivre l’algorithme suivant :

---

**Algorithm 2** Algorithme de Viterbi

---

**Require:** Matrice de transition  $A = (a_{ij})_{1 \leq i, j \leq N}$ , probabilités d'émission  $B = (b_i(y))_{1 \leq i \leq N}$ , distribution initiale  $\pi = (\pi_i)_{1 \leq i \leq N}$ , suite d'observations  $Y = (y_1, \dots, y_T)$

**Ensure:** Chemin d'états le plus probable  $X^* = (X_1^*, \dots, X_T^*)$  et probabilité associée  $P^*$

**Initialisation**

```
1: for  $i = 1, \dots, N$  do  
2:    $\delta_1(i) \leftarrow \pi_i b_i(y_1)$   
3:    $\psi_1(i) \leftarrow 0$   
4: end for
```

**Récurrance**

```
5: for  $t = 2, \dots, T$  do  
6:   for  $j = 1, \dots, N$  do  
7:      $\delta_t(j) \leftarrow \left[ \max_{1 \leq i \leq N} \delta_{t-1}(i) a_{ij} \right] b_j(y_t)$   
8:      $\psi_t(j) \leftarrow \arg \max_{1 \leq i \leq N} \delta_{t-1}(i) a_{ij}$   
9:   end for  
10: end for
```

**Fin**

```
11:  $P^* \leftarrow \max_{1 \leq i \leq N} \delta_T(i)$   
12:  $X_T^* \leftarrow \arg \max_{1 \leq i \leq N} \delta_T(i)$ 
```

**Retracage du chemin**

```
13: for  $t = T - 1, T - 2, \dots, 1$  do  
14:    $X_t^* \leftarrow \psi_{t+1}(X_{t+1}^*)$   
15: end for  
16: return  $X^*, P^*$ 
```

---

On obtient ainsi avec  $X_1^*, \dots, X_T^*$  notre séquence d'état recherchée.

**Analyse de la complexité :**

- A chaque instant, on calcule une valeur pour chacun des  $N$  états.
- Pour chaque état, on doit évaluer la probabilité de venir depuis chacun des états.

Ainsi, à chaque itération, on a une complexité de  $O(N^2)$ .

Il y a  $T$  instant, on obtient donc une complexité temporelle totale de :

$$O(TN^2)$$

### 3.3 Estimation et Algorithme de Baum–Welch

Ce dernier problème est celui qui nous intéressera le plus dans les applications économiques. Nous allons chercher à estimer l'ensemble des paramètres de notre modèle, la distribution initiale, les probabilités de changement d'états ainsi que les paramètres des différentes lois des observations, et tout ça uniquement à partir d'une série d'observations. C'est-à-dire qu'on cherche le modèle qui maximise la probabilité de voir ce qu'on a observé.

i.e.

$$\operatorname{argmax}_{(A,B,\pi)} \mathbb{P}(Y \mid (A, B, \pi))$$

Pour cela, nous pouvons utiliser un algorithme introduit par Baum–Welch qui est en fait un cas particulier d'utilisation de l'algorithme Expectation Maximisation (EM).

Avant toutes choses, il est important de noter que cette méthode itérative permet d'approximer un optimal local à notre problème.

Afin de pouvoir trouver les  $A$ ,  $B$  et  $\pi$  de notre problème, nous devons définir les variables avec lesquelles nous allons travailler :

- $\xi_t(i, j) :=$  la probabilité d'être en l'état  $i$  à l'instant  $t$  et en l'état  $j$  à l'instant  $t + 1$  sachant le modèle  $(A, B, \pi)$  et la série d'observations  $Y$ .

$$\forall t \in \{1, \dots, T - 1\}, \quad \forall i, j \in E,$$

$$\begin{aligned} \xi_t(i, j) &= \mathbb{P}(X_t = i, X_{t+1} = j \mid Y, (A, B, \pi)) \\ &= \frac{\mathbb{P}(X_t = i, X_{t+1} = j, Y \mid (A, B, \pi))}{\mathbb{P}(Y \mid A, B, \pi)} \\ &= \frac{\alpha_t(i) a_{ij} b_j(y_{t+1}) \beta_{t+1}(j)}{\mathbb{P}(Y \mid A, B, \pi)} \\ &= \frac{\alpha_t(i) a_{ij} b_j(y_{t+1}) \beta_{t+1}(j)}{\sum_{k \in E} \sum_{\ell \in E} \alpha_t(k) a_{k\ell} b_\ell(y_{t+1}) \beta_{t+1}(\ell)} \end{aligned}$$



- $\gamma_t(i) :=$  la probabilité d'être en l'état  $i$  à l'instant  $t$ , sachant la série d'observation et le modèle  $(A, B, \pi)$ .

$$\forall t \in \{1, \dots, T-1\}, \quad \forall i \in E,$$

$$\begin{aligned} \gamma_t(i) &= \mathbb{P}(X_t = i \mid Y, (A, B, \pi)) \\ &= \sum_{j \in E} \mathbb{P}(X_t = i, X_{t+1} = j \mid Y, (A, B, \pi)) \\ &= \sum_{j \in E} \xi_t(i, j) \end{aligned}$$

On peut remarquer qu'en sommant les  $\gamma_t(i)$  par rapport aux instants  $t$ , on obtient le nombre de fois où l'état  $i$  est visité, ou de manière équivalente, l'espérance du nombre de transitions depuis  $i$ .

De la même manière, la somme sur les  $t$  des  $\xi_t(i, j)$  donne l'espérance du nombre de transitions de l'état  $i$  vers l'état  $j$ .

i.e.

$$\begin{aligned} \sum_{t=1}^T \gamma_t(i) &= \text{espérance du nombre de transitions depuis } i \\ \sum_{t=1}^{T-1} \xi_t(i, j) &= \text{espérance du nombre de transitions de } i \text{ vers } j \end{aligned}$$

Grâce à cette remarque, des estimateurs naturels de  $\pi$ ,  $A$ ,  $B$  apparaissent :

$$\forall i, j \in E,$$

$$\pi'_i := \text{probabilité d'être en l'état } i \text{ au moment } 1 = \gamma_1(i)$$

$$a'_{ij} := \frac{\text{espérance du nombre de transitions de l'état } i \text{ à } j}{\text{espérance du nombre de transitions depuis l'état } i} = \frac{\sum_{t=1}^{T-1} \xi_t(i, j)}{\sum_{t=1}^{T-1} \gamma_t(i)}$$

$$\begin{aligned} b'_j(y) &:= \frac{\text{espérance du nombre d'observations de } y \text{ étant dans l'état } j}{\text{espérance du nombre de fois où on est en l'état } j} \\ &= \frac{\sum_{t=1}^T \gamma_t(j) \mathbf{1}_{\{y_t=y\}}}{\sum_{t=1}^T \gamma_t(j)} \end{aligned}$$

Dans le cas où les observations sont continues, alors on estime les paramètres avec des estimateurs naturels issues de ces quantités.

Par exemple pour des émissions gaussiennes, on a :

$$\mu_j = \frac{\sum_{t=1}^T \gamma_t(j) y_t}{\sum_{t=1}^T \gamma_t(j)}$$

et

$$\sigma_j^2 = \frac{\sum_{t=1}^T \gamma_t(j) (y_t - \mu_j)^2}{\sum_{t=1}^T \gamma_t(j)}$$

On peut donc en déduire cet algorithme :

---

**Algorithm 3** Algorithme de Baum–Welch

---

**Require:** Suite d'observations  $Y = (y_1, \dots, y_T)$ ,

- 1: nombre d'états  $N$ ,
- 2: paramètres initiaux  $\lambda^{(0)} = (A^{(0)}, B^{(0)}, \pi^{(0)})$ ,
- 3: seuil de convergence  $\varepsilon$

**Ensure:** Paramètres estimés  $\hat{\lambda} = (\hat{A}, \hat{B}, \hat{\pi})$

- 4: Initialiser  $m \leftarrow 0$

5: **repeat**

**Étape E : estimation des variables cachées**

- 6:   Pour  $i = 1, \dots, N$  :

$$\alpha_1(i) = \pi_i^{(m)} b_i^{(m)}(y_1)$$

- 7:   Pour  $t = 2, \dots, T$  et  $j = 1, \dots, N$  :

$$\alpha_t(j) = b_j^{(m)}(y_t) \sum_{i=1}^N \alpha_{t-1}(i) a_{ij}^{(m)}$$

- 8:   Pour  $i = 1, \dots, N$  :

$$\beta_T(i) = 1$$

- 9:   Pour  $t = T - 1, \dots, 1$  et  $i = 1, \dots, N$  :

$$\beta_t(i) = \sum_{j=1}^N a_{ij}^{(m)} b_j^{(m)}(y_{t+1}) \beta_{t+1}(j)$$

- 10:   Pour  $t = 1, \dots, T$  et  $i = 1, \dots, N$  :

$$\gamma_t(i) = \frac{\alpha_t(i) \beta_t(i)}{\sum_{k=1}^N \alpha_t(k) \beta_t(k)}$$

- 11:   Pour  $t = 1, \dots, T - 1$  et  $i, j = 1, \dots, N$  :

$$\xi_t(i, j) = \frac{\alpha_t(i) a_{ij}^{(m)} b_j^{(m)}(y_{t+1}) \beta_{t+1}(j)}{\sum_{k=1}^N \sum_{\ell=1}^N \alpha_t(k) a_{k\ell}^{(m)} b_{\ell}^{(m)}(y_{t+1}) \beta_{t+1}(\ell)}$$

**Étape M : mise à jour des paramètres**

- 12:   Pour  $i = 1, \dots, N$  :

$$\pi_i^{(m+1)} = \gamma_1(i)$$

- 13:   Pour  $i, j = 1, \dots, N$  :

$$a_{ij}^{(m+1)} = \frac{\sum_{t=1}^{T-1} \xi_t(i, j)}{\sum_{t=1}^{T-1} \gamma_t(i)}$$

- 14:   Mettre à jour les paramètres d'émission  $B^{(m+1)}$  avec des estimateurs naturels basés sur les  $\gamma_t(i)$

- 15:    $L^{(m+1)} \leftarrow \sum_{i=1}^N \alpha_T(i)$

- 16:    $m \leftarrow m + 1$

- 17: **until**  $|L^{(m)} - L^{(m-1)}| < \varepsilon$

- 18: **return**  $\hat{\lambda} = \lambda^{(m)}$
-

Dans ce travail, nous ne traiterons pas du choix de epsilon qui mériterait un travail à part entière, mais il est important de noter que ce choix peut avoir un impact significatif sur la convergence de l'algorithme. De plus, avec une seule série d'observations, on ne peut pas estimer de manière robuste la distribution initiale  $\pi$ , car elle ne concerne que l'état caché au premier instant ; toutefois, cela a peu d'impact sur l'utilisation finale du modèle, puisque l'influence de  $\pi$  disparaît progressivement au fil du temps au profit des transitions et des émissions estimées.

### Analyse de la complexité :

- étape forward : pour chaque instant  $t$ , pour chaque état  $j$ , on somme sur tous les états précédents. Donc  $O(TN^2)$ .
- étape backward : idem que pour forward. Donc  $O(TN^2)$ .
- calcul des  $\gamma_t(i)$  : calcul utilisant  $\alpha_t(i)$  et  $\beta_t(i)$ , sur chaque  $t$  et chaque  $i$ . Donc  $O(TN)$ .
- calcul des  $\xi_t(i, j)$  : calcul utilisant  $\alpha_t(i)$  et  $\beta_t(j)$  pour chaque  $t$  et chaque couple  $(i, j)$ . Donc  $O(TN^2)$ .
- mise à jour des  $\pi_i$  : on récupère la valeur de  $\gamma_1(i)$  pour chaque  $i$ . Donc  $O(N)$ .
- mise à jour des  $a_{ij}$  : calcul avec une somme sur  $T$  pour chaque couple  $(i, j)$ . Donc  $O(TN^2)$ .
- mise à jour des  $b_j(k)$  : si on boucle naïvement sur tous les symboles  $k$ , les états  $j$  et les instants  $t$ , alors  $O(TNM)$ . Or on peut parcourir les observations efficacement et obtenir une complexité de  $O(TN)$ .

Ainsi, on obtient une complexité globale par itération de :

$$O(TN^2)$$

Si l'algorithme fait  $I$  itérations avant convergence, alors on a une complexité totale de :

$$O(ITN^2)$$

## 4 Limites des algorithmes et analyse de la qualité des estimations

### 4.1 Les limites des algorithmes

### 4.2 Limites et Mesure de la qualité de la vraisemblance avec l'algorithme Forward

#### 4.2.1 Limites de l'algorithme de forward

Une des principales limites de l'algorithme de forward vient de la quantité d'observations. En effet, plus la séquence d'observations est longue, plus la vraisemblance  $\mathbb{P}(Y)$  devient extrêmement petite, ce qui peut entraîner des problèmes computationnels liés à la précision numérique. Pour pallier à ce problème, on peut alors effectuer une modification de l'algorithme de forward en passant à la log-vraisemblance. Cela permet de transformer les produits en sommes, le tout avec des valeurs plus grandes et par conséquent plus faciles à manipuler numériquement.

De plus, si le nombre d'états cachés est très grand, l'algorithme de forward peut devenir extrêmement coûteux en temps de calcul, ce qui peut rendre son utilisation moins pertinente dans des applications avec un grand nombre d'états.

Aussi, l'algorithme Forward évalue la vraisemblance des observations, mais il ne dit pas directement si les régimes cachés estimés sont économiquement interprétables. Un modèle peut avoir une bonne vraisemblance tout en produisant des régimes difficiles à lire financièrement. Ainsi, la vraisemblance calculée par Forward doit être utilisée comme une mesure statistique importante, mais elle doit être complétée par l'analyse des régimes obtenus avec Viterbi ou avec les probabilités a posteriori, ainsi que par une validation hors échantillon.

L'algorithme Forward permet donc de calculer la probabilité d'observer une séquence donnée sous un modèle HMM donné. Si  $Y_{1:T}$  désigne la série observée, avec  $T$  le nombre total d'observations, et si  $\lambda = (\pi, A, B)$  désigne les paramètres du modèle, l'algorithme Forward calcule :

$$P(Y_{1:T} \mid \lambda).$$

En pratique, on utilise plutôt la log-vraisemblance :

$$\ell(\lambda) = \log P(Y_{1:T} \mid \lambda),$$

car la vraisemblance brute devient très petite lorsque  $T$  est grand.

### 4.2.2 Interprétation de la log-vraisemblance

La log-vraisemblance mesure la capacité du modèle à expliquer les observations. Plus elle est élevée, plus le modèle attribue une forte probabilité à la série observée. Elle constitue donc une première mesure de qualité statistique du HMM.

Cependant, cette mesure doit être interprétée avec prudence. Un modèle plus complexe, par exemple avec davantage d'états cachés, peut obtenir une meilleure log-vraisemblance simplement parce qu'il possède plus de paramètres. Une bonne log-vraisemblance ne signifie donc pas nécessairement que les régimes estimés sont plus interprétables, ni que le modèle restera performant sur de nouvelles observations.

### 4.2.3 Comparaison entre modèles

Pour comparer plusieurs HMM, on peut utiliser la log-vraisemblance moyenne par observation :

$$\bar{\ell}(\lambda) = \frac{1}{T} \log P(Y_{1:T} \mid \lambda).$$

Cette normalisation permet de comparer des séries de longueurs différentes.

On peut aussi utiliser des critères pénalisés comme l'AIC et le BIC :

$$AIC = -2\ell(\lambda) + 2p,$$

$$BIC = -2\ell(\lambda) + p \log(T),$$

où  $p$  désigne le nombre de paramètres estimés. Ces critères pénalisent les modèles trop complexes. Dans ce cas, le meilleur modèle est celui qui minimise l'AIC ou le BIC.

### 4.2.4 Évaluation hors échantillon

Une bonne log-vraisemblance sur l'échantillon d'apprentissage ne suffit pas. Le modèle peut très bien expliquer les données passées tout en étant moins performant sur des données nouvelles. Pour une série temporelle financière, on découpe donc le jeu de données dans l'ordre chronologique : une première partie sert à l'apprentissage, puis la partie qui suit sert au test. Il faut donc évaluer la vraisemblance sur cette période de test, qui n'a pas servi à l'estimation.

Si le modèle  $\hat{\lambda}_{\text{train}}$  est estimé sur  $Y_{1:T_{\text{train}}}$ , et si  $Y_{\text{test}}$  désigne les observations de la période test, on peut calculer :

$$\ell_{\text{test}} = \log P(Y_{\text{test}} \mid \hat{\lambda}_{\text{train}}).$$

On utilise souvent la log-vraisemblance moyenne de test :

$$\bar{\ell}_{\text{test}} = \frac{1}{T_{\text{test}}} \ell_{\text{test}},$$

ou encore la negative log predictive density :

$$NLPD = -\frac{1}{T_{\text{test}}} \ell_{\text{test}}.$$

Un  $NLPD$  faible indique que le modèle attribue une forte probabilité aux nouvelles observations. Il s'agit donc d'un bon indicateur pour vérifier si le modèle reste pertinent sur des données qui n'ont pas servi à l'estimation.

### 4.3 Mesure de la qualité du décodage par Viterbi

L'algorithme de Viterbi permet d'estimer la séquence d'états cachés la plus probable compte tenu des observations et des paramètres du modèle. Si  $Y_{1:T}$  désigne la série observée, avec  $T$  le nombre total d'observations, et  $\hat{\lambda}$  les paramètres estimés du modèle, Viterbi cherche le chemin

$$\hat{X}_{1:T}^V \in \arg \max_{x_{1:T}} P(X_{1:T} = x_{1:T} \mid Y_{1:T}, \hat{\lambda}).$$

Dans une application financière, ce chemin peut être interprété comme une chronologie des régimes de marché : chaque date est associée à un régime estimé, par exemple calme, volatil, stressé ou en reprise.

#### 4.3.1 Difficulté de l'évaluation sur données réelles

La difficulté principale est que, sur données financières réelles, les vrais états cachés ne sont pas observés. On ne dispose donc pas d'une séquence vraie à laquelle comparer directement la séquence estimée. Il est alors difficile de calculer des mesures quantitatives classiques comme une accuracy, une matrice de confusion ou un taux d'erreur.

L'évaluation de Viterbi est donc surtout indirecte. Un chemin Viterbi très net n'est pas nécessairement le vrai chemin des états cachés : c'est seulement le chemin le plus probable sous le modèle estimé.

#### 4.3.2 Interprétation économique des régimes détectés

Sur données réelles, un premier critère consiste à comparer les régimes détectés à des périodes financières connues. Par exemple, un régime interprété comme un régime de stress devrait apparaître davantage pendant des crises, des krachs ou des périodes de forte volatilité. À l'inverse, un régime calme devrait dominer pendant des périodes plus régulières.

Cette lecture reste qualitative, mais elle est importante. Si les états détectés ne correspondent à aucune dynamique financière claire, le décodage est difficile à exploiter.

On peut aussi vérifier la persistance des régimes. Un chemin qui change d'état presque à chaque date peut indiquer que le modèle capte surtout du bruit. Le chemin doit donc être assez stable pour

être interprétable.

### 4.3.3 Comparaison avec le posterior decoding

Viterbi ne choisit pas l'état le plus probable à chaque date séparément. Il choisit le chemin globalement le plus probable, c'est-à-dire qu'il maximise :

$$P(X_{1:T} = x_{1:T} \mid Y_{1:T}, \hat{\lambda}).$$

Cette contrainte donne un chemin cohérent temporellement, mais elle peut conduire à choisir, à certaines dates, un état qui n'est pas celui ayant la plus forte probabilité marginale.

Le posterior decoding adopte une logique différente. Il calcule les probabilités a posteriori :

$$\hat{\gamma}_t(i) = P(X_t = i \mid Y_{1:T}, \hat{\lambda}),$$

puis choisit, à chaque date  $t$ , l'état marginalement le plus probable :

$$\hat{X}_t^P \in \arg \max_{i \in E} \hat{\gamma}_t(i).$$

Ainsi, le posterior decoding prend une décision locale date par date, tandis que Viterbi optimise une trajectoire complète.

Comparer les deux approches est intéressant, car elles ne répondent pas exactement à la même question. Si Viterbi et posterior decoding donnent des classifications proches, cela indique que les régimes sont bien identifiés par le modèle.

Un exemple simple de mesure d'écart est le taux de désaccord entre les deux décodages :

$$D_{VP} = \frac{1}{T} \sum_{t=1}^T \mathbf{1}_{\{\hat{X}_t^V \neq \hat{X}_t^P\}}.$$

Plus  $D_{VP}$  est proche de 0, plus les deux méthodes donnent des régimes similaires. Une valeur élevée indique au contraire que le choix des états est incertain ou dépend fortement de la méthode de décodage.

À l'inverse, si les deux méthodes donnent des résultats très différents, cela révèle une incertitude importante sur les états cachés. Le chemin Viterbi doit alors être interprété comme une trajectoire probable, et non comme une classification certaine.



## 4.4 Mesure de la qualité des estimations de Baum–Welch

Baum–Welch estime les paramètres d’une chaîne de Markov cachée à partir des seules observations  $Y_{1:T}$ . Si  $X_t$  désigne l’état caché à l’instant  $t$ , le modèle est donné par :

$$\lambda = (\pi, A, B)$$

avec

$$\pi_i = P(X_1 = i), \quad a_{ij} = P(X_{t+1} = j \mid X_t = i)$$

et  $B$  l’ensemble des lois d’émission  $b_i(y)$ , ou plus généralement  $b_i(y; \theta_i)$  lorsque ces lois dépendent de paramètres  $\theta_i$ . L’algorithme cherche alors

$$\hat{\lambda} \in \arg \max_{\lambda} P(Y_{1:T} \mid \lambda).$$

Comme il s’agit d’une méthode de type EM, la solution obtenue est en général un optimum local et pas nécessairement le meilleur modèle possible. La qualité des estimations doit donc être étudiée à travers la convergence, la stabilité des paramètres, l’interprétation économique et la performance hors échantillon.

### 4.4.1 Convergence de la log-vraisemblance

À l’itération  $k$ , on note  $\ell^{(k)} = \log P(Y_{1:T} \mid \lambda^{(k)})$ . Ici,  $T$  est la longueur de la série observée, c’est-à-dire le nombre total d’observations. Ainsi, les quantités divisées par  $T$  sont des gains moyens par observation.

Baum–Welch doit produire une log-vraisemblance non décroissante. On suit donc :

$$\Delta_k = \ell^{(k)} - \ell^{(k-1)}, \quad \overline{\Delta}_k = \frac{\Delta_k}{T}, \quad r_k = \frac{\Delta_k}{1 + |\ell^{(k-1)}|}.$$

Un critère d’arrêt raisonnable est  $r_k < 10^{-6}$ , ou bien  $\overline{\Delta}_k < 10^{-6}$  à  $10^{-5}$ , pendant plusieurs itérations consécutives. Ces seuils ne doivent pas être vus comme des règles absolues, mais comme des ordres de grandeur permettant de repérer le moment où l’amélioration devient négligeable.

Une convergence trop rapide peut être suspecte si l’algorithme s’arrête en moins de cinq itérations avec des paramètres proches de l’initialisation. Ce cas n’est pas forcément mauvais, mais il doit être comparé aux résultats obtenus avec d’autres initialisations. En lançant Baum–Welch  $M$  fois, on compare :

$$\frac{\ell_{\max}^* - \ell_m^*}{T}, \quad \ell_{\max}^* = \max_{1 \leq m \leq M} \ell_m^*.$$

Un écart de  $10^{-3}$  à  $10^{-2}$  par observation peut signaler une solution nettement moins bonne. À l’inverse, une convergence instable apparaît lorsque  $\Delta_k < 0$ . Une petite baisse numérique peut être tolérée si  $\Delta_k/T > -10^{-8}$ , mais des baisses plus fortes ou répétées indiquent un problème de

normalisation, de sous-flux numérique ou d'implémentation.

#### 4.4.2 Stabilité des paramètres estimés

Comme Baum–Welch dépend de l'initialisation, il faut répéter l'estimation avec plusieurs points de départ. Avant de comparer les résultats, on aligne les états afin de corriger le changement d'étiquette : l'état 1 d'une estimation peut correspondre à l'état 2 d'une autre. Pour deux estimations (a) et (b), on peut mesurer :

$$D_\pi = \frac{1}{2} \sum_i \left| \hat{\pi}_i^{(a)} - \hat{\pi}_i^{(b)} \right|, \quad D_A = \frac{1}{N} \sum_i \frac{1}{2} \sum_j \left| \hat{a}_{ij}^{(a)} - \hat{a}_{ij}^{(b)} \right|.$$

On compare également la durée moyenne des régimes, pour  $\hat{a}_{ii} < 1$  :

$$\hat{d}_i = \frac{1}{1 - \hat{a}_{ii}}, \quad D_{d,i} = \left| \frac{\hat{d}_i^{(a)} - \hat{d}_i^{(b)}}{\hat{d}_i^{(b)}} \right|.$$

Deux estimations peuvent avoir des vraisemblances proches mais des régimes de persistances très différentes.

Pour les lois d'émission discrètes, on peut utiliser :

$$D_B = \frac{1}{N} \sum_i \frac{1}{2} \sum_{y \in \mathcal{Y}} \left| \hat{b}_i^{(a)}(y) - \hat{b}_i^{(b)}(y) \right|.$$

Pour des lois continues de densités  $\hat{f}_i^{(a)}$  et  $\hat{f}_i^{(b)}$ , on peut utiliser une divergence de Kullback–Leibler :

$$\text{KL}(\hat{f}_i^{(a)} \parallel \hat{f}_i^{(b)}) = \int \hat{f}_i^{(a)}(y) \log \left( \frac{\hat{f}_i^{(a)}(y)}{\hat{f}_i^{(b)}(y)} \right) dy.$$

Si les émissions sont paramétrées par  $\theta_i$ , on compare les vecteurs  $\hat{\theta}_i$ , idéalement avec une distance normalisée afin de tenir compte des différences d'échelle entre les composantes.

#### 4.4.3 Interprétation économique des régimes

Une estimation satisfaisante doit produire des régimes économiquement lisibles. Les lois d'émission associées aux états doivent décrire des comportements distincts. Si deux états ont des lois presque identiques, le modèle utilise probablement trop d'états ou ne sépare pas clairement les régimes. La matrice de transition doit aussi être cohérente : dans les séries financières, les coefficients diagonaux  $a_{ii}$  ne doivent pas être trop faibles, car les régimes de marché présentent souvent une certaine persistance.

Les probabilités a posteriori

$$\hat{\gamma}_t(i) = P(X_t = i \mid Y_{1:T}, \hat{\lambda})$$

permettent de vérifier à quelles dates chaque régime domine. Un régime interprétable doit correspondre à des périodes financières cohérentes, par exemple des périodes de stress, de calme ou de forte incertitude.

#### 4.4.4 Validation hors échantillon

On parle aussi d'évaluation hors échantillon, de test hors échantillon, d'évaluation sur période de test ou encore d'évaluation prédictive sur données futures. Dans un contexte financier, le terme backtesting est également utilisé lorsque l'on teste une méthode sur une période passée qui n'a pas servi à l'estimation.

L'idée est d'estimer le modèle sur une période d'apprentissage puis de l'évaluer sur une période future non utilisée pour l'estimation. Si  $\hat{\lambda}_{train}$  est estimé sur  $Y_{1:T_{train}}$ , on calcule :

$$\ell_{test} = \log P(Y_{T_{train}+1:T} \mid \hat{\lambda}_{train}), \quad NLPD = -\frac{1}{T_{test}} \ell_{test}.$$

Un  $NLPD$  faible indique que le modèle attribue une forte probabilité aux nouvelles observations. Cette étape est importante, car une estimation peut bien expliquer le passé tout en généralisant mal sur des données financières futures.